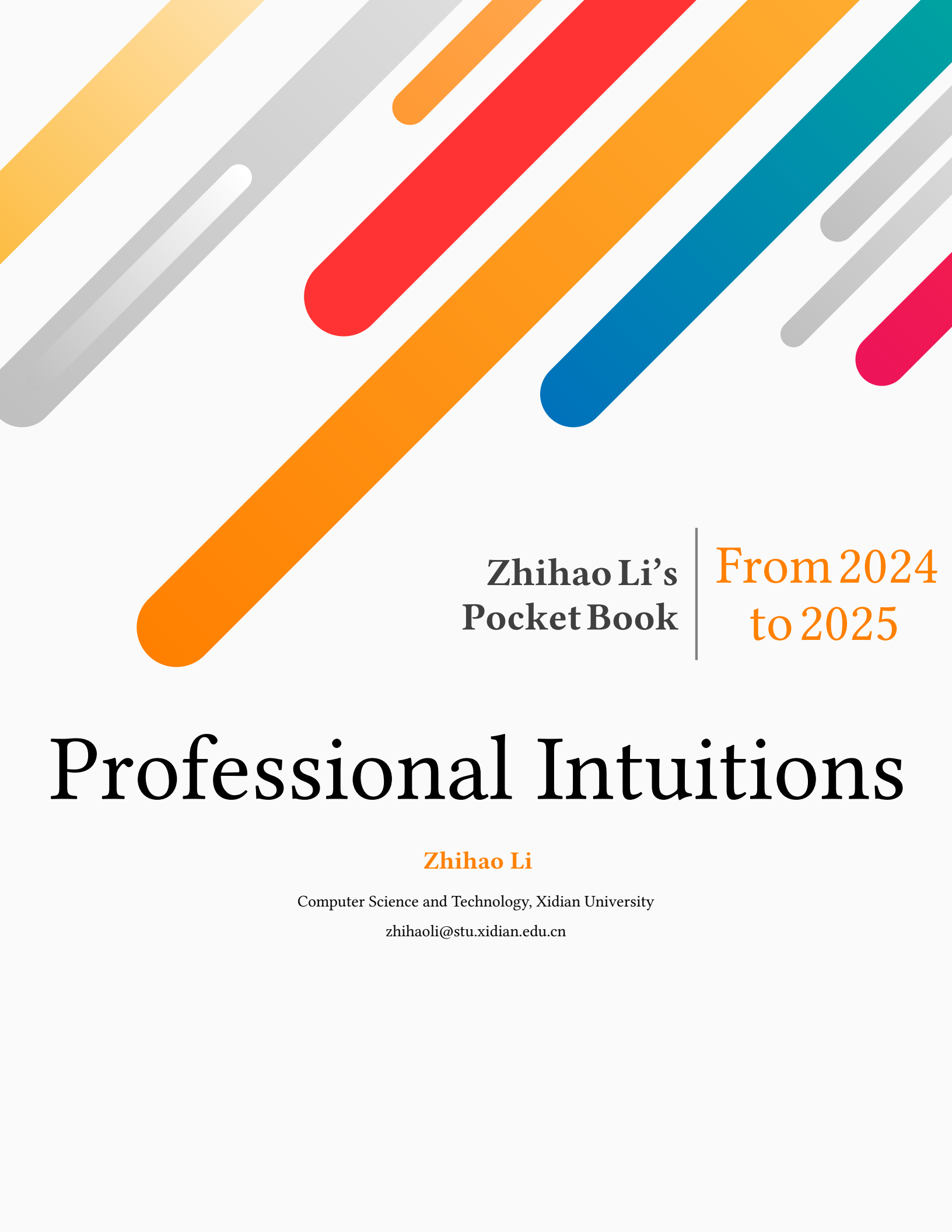




Zhihao Li's
Pocket Book

From 2024
to 2025

Professional Intuitions

Zhihao Li

Computer Science and Technology, Xidian University

zhihaoli@stu.xidian.edu.cn

CONTENTS

CHAPTER 1	PROFESSION & LEARNING	PAGE 4
1.1	Mathematical Principles	4
1.2	Computer Graphics	4
1.3	Digital Human	4
CHAPTER 2	PHILOSOPHY & INSIGHT	PAGE 5
2.1	Research Guidelines	5
2.1.1	Research Purpose	5
2.1.2	Global Stages	5
2.1.3	Early Stage	5
2.2	Research Habits	6
2.2.1	Paper Reading	6
2.2.2	Experiment Exploration	7
2.2.3	Balanced Life	7
2.2.4	Future Planning	7
CHAPTER 3	VIEWS & INSPIRATION	PAGE 8
3.1	Learning With Noisy Labels	8
3.2	Portrait Animation	8
3.2.1	Video Animation	8
3.2.2	Audio Animation	9
CHAPTER 4	CODING & SKILLS	PAGE 11
4.1	Environment Configuration	11
4.1.1	Miniconda Installation On Linux	11
4.1.2	Pytorch3d Installation on Windows	11
4.1.3	Ninja Installation on Linux	12
4.2	Python Debugging	12
4.2.1	Visual Studio Code Debugging Configuration	12
4.3	Bugs with Solution	13
4.3.1	Version Conflict	13

4.4	Coding Tricks	14
4.4.1	Python	14
4.4.2	Cmd On Windows	14

CHAPTER 5

GRAMMAR & WRITINGS

PAGE 15

5.1	Mathematical Symbols in LaTeX	15
5.1.1	集合及向量空间的表示	15
5.1.2	运算符号	15
5.2	Extended Markdown	15
5.3	Elaborating Ideas	16

CHAPTER 6

TEMPLATES & TOOLS

PAGE 17

Chapter 1

Profession & Learning

1.1 Mathematical Principles

1. 仿射变换 (Affine Transformation)

$$price = w_{area} \cdot area + w_{age} \cdot age + b \quad (1.1)$$

仿射变换的特点是通过加权和对特征进行线性变换，并通过偏置项进行平移。

2. 非线性频率压缩

在滤波器设计中将整个模拟频率轴压缩到 π/T 之间，使得 $H_a(s), s = j\Omega$ 压缩为 $\widehat{H}_a(s_1), s_1 = j\Omega_1$ ，可以利用正切变换实现频率压缩模型：

$$\Omega = \frac{2}{T} \tan\left(\frac{1}{2}\Omega_1 T\right) \quad (1.2)$$

这个设计思想实质上利用了正切函数定义域有限、值域无限以及奇函数的性质；推而广之，这种设计可以实现特定的单值压缩方法，也可以实现值域的延展。

一些类似的函数特性，对数函数，指数函数分别适合于定义域、值域取值 $0 \sim 1$ 之间的情况，但是对目标域都有所限制，因此这些函数往往没有正切函数具有优良的特性。

1.2 Computer Graphics

1.3 Digital Human

Chapter 2

Philosophy & Insight

2.1 Research Guidelines

2.1.1 Research Purpose

“大道至简”，追求 *simple and effective* 研究。学习姚期智院士的选题三问，

1. 你的研究选题真的很重要吗？
2. 有多少人在做？
3. 为什么轮到了你来做？

关于学术品味，姚期智院士曾讲，“一个好的问题应该是漂亮的，即简洁、新颖、令人惊讶；也应该是重要的，即人们关心其答案；并且应该具有普遍吸引力，能够超越学术界限，引起广泛关注”。

- Attractive (漂亮的): Simplicity; Novelty; Surprise;
- Importance (重要的): People care what the answer is;
- Universe (普遍的): Transcends academic boundaries.

2.1.2 Global Stages

1. 不要被工程化的思维所束缚，做 A+B 的工作，着重分析本质，从源头创新。

2.1.3 Early Stage

1. 前期注重多看研究论文，精度并总结，打牢基础，做研究综述。
2. **研究综述**：有多少途径能够实现研究目的，各自是从哪些地方出发的，本质的区别是什么。

2.2 Research Habits

2.2.1 Paper Reading

跟进前沿

关注领域内发展，要跟进最新研究成果，及时搜集最新研究论文。

- 统一的渠道：X-MOL 学术平台并保持邮件推送。
- 分散的渠道：各个期刊会议的官网。

论文略读

1. 阅读题目和整体流程图；
 2. 阅读摘要和 Introduction；
 3. 选择性阅读 Method 模块。
- 阅读原则：明确研究目标、研究问题、解决方案以及方法流程。
 - 总结原则：一段话总结 + 贡献点 + 启发思路。

论文精读

1. 先阅读题目、摘要、Introduction 和 Method；
 2. 再阅读 Related Work；
 3. 选择性阅读实验部分。
- 阅读原则：明确研究目标、研究问题、解决方案以及方法流程。
 - 总结原则：一段话总结 + 手推思路。

论文总结

1. 论文阅读后及时总结，多发掘创新点、启发点；
 2. 积累论文写作经验，可以在阅读论文中总结；
 3. 集中在一个地方整理笔记，形成研究发展体系。
- 略读选择的平台：TexStudio 汇总笔记。
 - 精读选择的平台：TexStudio 汇总笔记 + 纸质笔记。

2.2.2 Experiment Exploration

1. 积累代码实现，技巧、工具库以及模板等；
 2. 多进行代码阅读，关注核心代码的编写；
 3. 关注重要的实验细节，积累调参技巧。
- 积累选择的平台：TexStudio 汇总笔记。

2.2.3 Balanced Life

1. 调控健康的生活节奏，尽量不要熬夜、不要懒觉；
2. 安排适量的运动，保证每周都能有三到五次的运动锻炼；
3. 注重健康饮食，忌高油高盐，保持均衡的营养摄入。

2.2.4 Future Planning

1. 提起规划未来半年到一年的安排；
2. 半年内的计划需清晰具象；
3. 一年内的计划可模糊笼统。

Chapter 3

Views & Inspiration

3.1 Learning With Noisy Labels

1. Co-teaching: Robust Training of Deep Neural Networks with Extremely Noisy Labels

- Views: DNN 的相互指导学习机制，两个模型分别动态地选取一些干净样本相互提供给对方进行学习，目的是过滤不同类型的噪声；
- Inspiration: 直觉是同辈相互交错的学习机制，而且训练中其样本选取的动态性值得一提；

2. DivideMix: Learning with Noisy Labels as Semi-supervised Learning

- Views: 对噪声数据集进行划分，并同时学习两个模型进行相互指导、标签集成，以克服不同类型的噪声；
- Inspiration: Mutual Learning 和交互学习一定程度上可以增强模型鲁棒性；

3.2 Portrait Animation

3.2.1 Video Animation

1. LivePortrait: Efficient Portrait Animation with Stitching and Retargeting Control

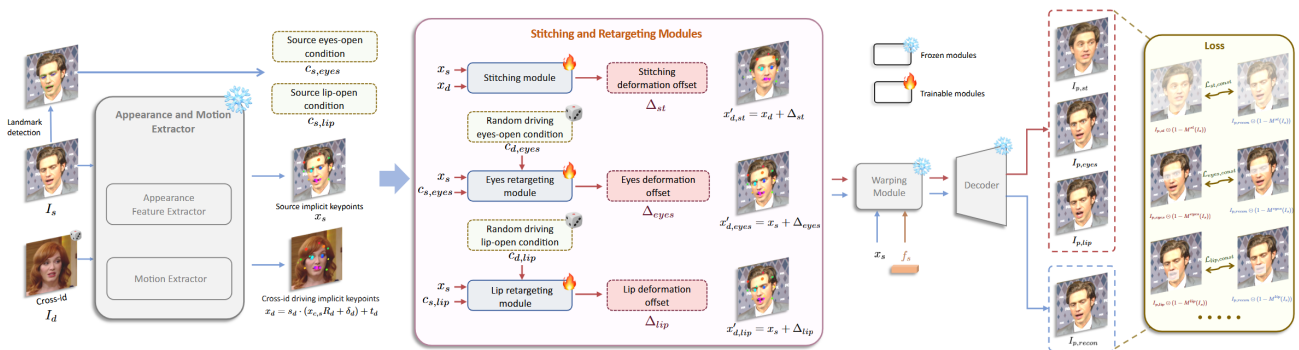


Figure 3. Pipeline of the second stage: stitching and retargeting modules training. After training the base model in the first stage, we freeze the appearance and motion extractor, warping module and decoder. Only the stitching module and the retargeting modules are optimized in the second stage. Please refer to Sec. 3.3 for details.

Figure 3.1: LivePortrait

- Views: 作者不同于主流基于扩散模型的方法, 提出了基于隐式关键点的视频驱动肖像动画框架, 能够明显地增强生成质量和泛化性能, 并利用小型 MLP 设计了一个先进的缝接模块和两个重定向模块实现平衡计算效率和可控性的目的。
- Roadmap: 基于 Face vid2vid, 核心模块是构建的形变场。
- Inspiration: 采用隐式关键点以及小型 MLP 给模型计算效率带来新的突破。

3.2.2 Audio Animation

1. TalkingGaussian: Structure-Persistent 3D Talking Head Synthesis via Gaussian Splatting

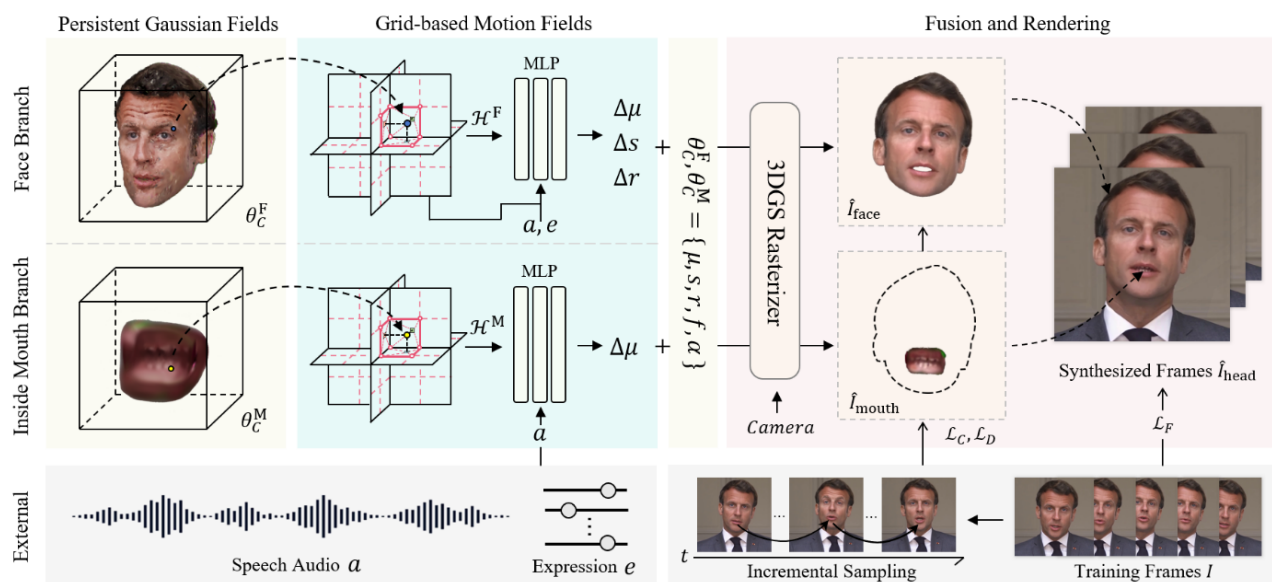


Fig. 2: Overview of TalkingGaussian. Learning from the speech video with training frames I , TalkingGaussian builds two separate branches to represent the dynamic face and inside mouth areas. Queried by the primitives in Persistent Gaussian Fields with parameters θ_C , a point-wise deformation can be predicted from Grid-based Motion Fields conditioned with audio feature $\mathbf{a}$ and upper-face expression $\mathbf{e}$. After that, the 3DGS rasterizer renders the deformed 3D Gaussian primitives into 2D images observed from the given camera, which are then fused to synthesize the entire talking head.

Figure 3.2: TalkingGaussian

- 研究目标: talking head synthesis
- 研究问题: 解决对迅速变化外观的不准确预测带来的固有的面部扭曲问题。
- 解决方案: 作者采用基于形变的网络, 对姿势一致性的人头结构施加预测的形变位移, 而不是直接预测形变后的人头, 并提出了一个 Face-Mouth 分解模块, 采用两个相互独立的网络分别学习人脸和嘴唇的形变。
- 启发思路: 施加预测的形变位移, 模块的分解能够提供更加准确的控制而且易于学习。
- 技术路线: 基于 3DGS 构造高斯球面, 首先学习一个基本的粗略平均场, 然后通过 MLP 预测对应的位移以获得形变后的形变场, 优化每个高斯球的属性实现最佳效果, 再利用 3DGS 的光栅化器,

计算逐像素的颜色，最终渲染出图像。

Chapter 4

Coding & Skills

4.1 Environment Configuration

4.1.1 Miniconda Installation On Linux

```
1  # Fetch the miniconda script
2  export HOME=$PWD
3  wget -q https://repo.anaconda.com/miniconda/ \
4  Miniconda3-py37_4.12.0-Linux-x86_64.sh -O miniconda.sh
5  sh miniconda.sh -b -p $HOME/miniconda3
6  rm miniconda.sh
7  export PATH=$HOME/miniconda3/bin:$PATH
8
9  # Initialize conda
10 source $HOME/miniconda3/etc/profile.d/conda.sh
11 hash -r
12 conda config --set always_yes yes --set changeps1 yes
13
14 # Create new environment
15 conda create -n my_env python=3.8
16 conda activate my_env
```

4.1.2 Pytorch3d Installation on Windows

Windows 下 3D Vision 环境配置 (GaussianAvatars)

4.1.3 Ninja Installation on Linux

```
1 apt install ninja-build
```

4.2 Python Debugging

4.2.1 Visual Studio Code Debugging Configuration

为了能够在 Visual Studio Code 中调试 Python 项目，首先需要安装 Python 和 Python Debugger 扩展。然后如图4.1所示，点击左侧的调试按钮，添加 launch.json 文件，并根据代码的配置示例创建项目配置。

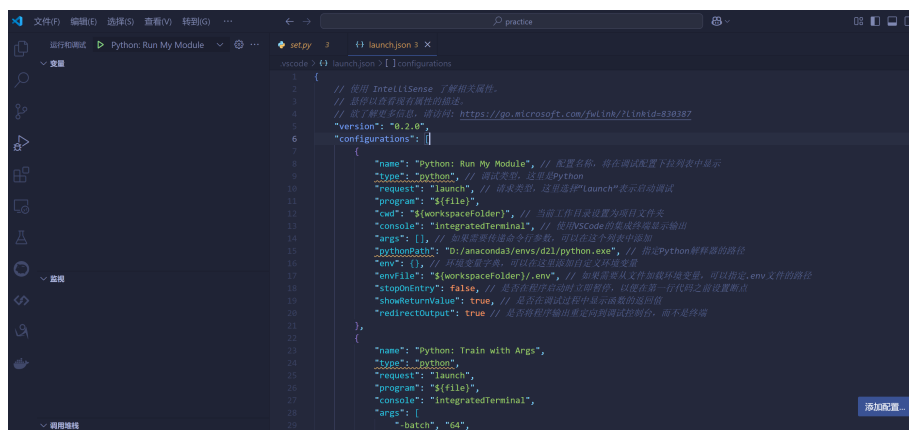


Figure 4.1: Python 项目调试配置

```
1 {
2     "version": "0.2.0",
3     "configurations": [
4         {
5             // 配置名称，将在调试配置下拉列表中显示
6             "name": "Python: Run My Module",
7             "type": "python", // 调试类型，这里是 Python
8             "request": "launch", // 请求类型，这里选择 "launch" 表示启动调试
9             "program": "${file}",
10            "cwd": "${workspaceFolder}", // 当前工作目录设置为项目文件夹
11            // 使用 VSCode 的集成终端显示输出
12            "console": "integratedTerminal",
13            "args": [
14                "--batch", "64",
15                "--epoches", "30",
16            ], // 如果需要传递命令行参数，可以在这个列表中添加
17            // 指定 Python 解释器的路径
```

```

18 "pythonPath": "D:/anaconda3/envs/d2l/python.exe",
19 "env": {}, // 环境变量字典，可以在这里添加自定义环境变量
20 // 如果需要从文件加载环境变量，可以指定.env 文件的路径
21 "envFile": "${workspaceFolder}/.env",
22 // 是否在程序启动时立即暂停，以便在第一行代码之前设置断点
23 "stopOnEntry": false,
24 "showReturnValue": true, // 是否在调试过程中显示函数的返回值
25 "redirectOutput": true // 是否将程序输出重定向到调试控制台，而不是终端
26 }
27 ]
28 }

```

在完成以上准备工作后，设置程序端点，然后直接点击调试三角形按钮，即可开始调试，如图 4.2 所示。

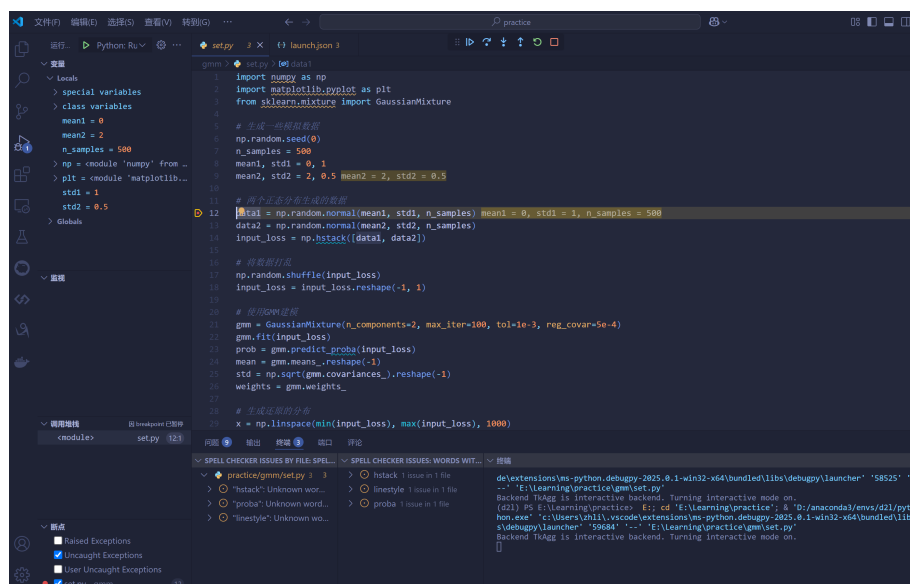


Figure 4.2: Python 程序调试示例

4.3 Bugs with Solution

4.3.1 Version Conflict

- Q: Numpy 版本和 Pytorch 版本不兼容。
- A: 尝试更换 Numpy 版本。

```

1 assert (self.faces==torch.from_numpy(flame_model.f.astype( \
2 'int64'))).all()
3 RuntimeError: Numpy is not available

```

2.
 - Q: PIL 版本更新问题。
 - A: 在 pillow 的 10.0.0 版本中,ANTIALIAS 方法被删除了,通过命令 `pip install Pillow==9.5.0` 安装旧版本即可。

```
1 AttributeError: module 'PIL.Image' has no attribute 'ANTIALIAS'
```

4.4 Coding Tricks

4.4.1 Python

1. 路径管理库 pathlib

- 类型指定: `input: Path`
- 路径赋值: `'../../dir'`
- 判断路径是否存在: `Path.exists()`
- 获取父目录: `Path.parent`
- 通配符匹配: `Path.glob()`

2. 数字可读性 8_000: Python 中数字表示含有下划线时, 仅用于提高大数字的可读性, 对数值没有影响。

4.4.2 Cmd On Windows

1. 打印当前目录路径 pwd

```
1 PS D:\ZHaoLi\02.Hobbies\01.Blog> pwd
2
3 Path
4 ----
5 D:\ZHaoLi\02.Hobbies\01.Blog
```

Chapter 5

Grammar & Writings

5.1 Mathematical Symbols in LaTeX

5.1.1 集合及向量空间的表示

Rendered Symbol	Source Code	Rendered Symbol	Source Code
S	<code>$\mathcal{S}$</code>	$\mathbb{R}$	<code>$\mathbb{R}$</code>

Table 5.1: LaTeX Symbols and Their Source Code

5.1.2 运算符号

Rendered Symbol	Source Code	Rendered Symbol	Source Code
$\ll$	<code>$\ll$</code>	$\mathbb{R}$	<code>$\mathbb{R}$</code>

Table 5.2: LaTeX Symbols and Their Source Code

5.2 Extended Markdown

在 Markdown 文件中实现下拉式列表，可以通过 ‘html’ 语言实现，具体语法如下：

```
1 <details>
2   <summary> Dependencies (click to expand) </summary>
3
4   ## Dependencies
5   - PyTorch 1.4
6   - matplotlib
7   - numpy
8   - imageio
```

- imageio-ffmpeg
- configargparse

The LLFF data loader requires ImageMagick.
</details>

5.3 Elaborating Ideas

1. 首先阐述问题的必要性，然后引出自己的方法。

Chapter 6

Templates & Tools